LLM Wiki - Deployment Guide

Prerequisites

Requirement	Version	Check Command
Node.js	>= 20 LTS	`node --version`
npm	>= 9	`npm --version`
TypeScript	>= 5.0 (dev)	`npx tsc --version`
Git	any	`git --version`

Provider-Specific Prerequisites

Provider	Prerequisite	How to Verify
Anthropic	Anthropic API key	Visit https://console.anthropic.com/settings/keys
Azure	Azure AI resource with deployed model	Azure Portal > AI Services > Keys and Endpoint
Vertex AI	Google Cloud project with Vertex AI API enabled, `gcloud` CLI installed	`gcloud auth application-default print-access-token`

Installation

From Source

```
# Clone the repository
git clone <repository-url>
cd wiki

# Install dependencies
npm install

# Verify build
npm run typecheck

# Build to dist/
npm run build
```

Global CLI Installation (optional)

```
# After building, link globally
npm link

# Now 'wiki' command is available system-wide
wiki --version
```

Initial Setup

1. Initialize a Wiki

```
# Create and navigate to your wiki directory
mkdir ~/my-wiki && cd ~/my-wiki

# Initialize the wiki structure
npx tsx /path/to/wiki/src/cli.ts init
# Or if globally linked:
wiki init
```

This creates:

```
my-wiki/
  config.json          # Configuration (edit this)
  .gitignore           # Excludes config.json, node_modules/
  sources/
    registry.json      # Source tracking
  wiki/
    index.md           # Content catalog
    log.md             # Action log
    sources/           # Source summaries
    entities/          # Entity pages
    topics/            # Topic pages
    synthesis/         # Cross-cutting analysis
    queries/           # Saved query results
  schema/
    wiki-schema.md     # LLM conventions
    prompts/           # Prompt templates (6 files)
```

2. Configure the LLM Provider

Edit `config.json` — it comes pre-populated with `_comment_*` fields that document each option and its environment variable.

Option A: Anthropic (Claude)

```
{
  "llm": {
    "provider": "anthropic",
    "model": "claude-sonnet-4-20250514",
    "apiKey": "",
    "maxTokens": 4096
  },
  "wiki": {
    "rootDir": "/absolute/path/to/my-wiki",
    "sourcesDir": "sources",
    "wikiDir": "wiki",
    "schemaDir": "schema"
  },
  "obsidian": {
    "enabled": true
  }
}
```

Then set the API key via environment variable:

```
export WIKI_LLM_API_KEY="sk-ant-api03-..."
```

Option B: Azure AI

```
{
  "llm": {
    "provider": "azure",
    "model": "gpt-4o",
    "apiKey": "",
    "azureEndpoint": "https://your-resource.openai.azure.com",
    "azureDeployment": "your-deployment-name",
    "maxTokens": 4096
  },
  "wiki": {
    "rootDir": "/absolute/path/to/my-wiki",
    "sourcesDir": "sources",
    "wikiDir": "wiki",
    "schemaDir": "schema"
  },
  "obsidian": {
    "enabled": true
  }
}
```

Then set the API key:

```
export WIKI_LLM_API_KEY="your-azure-api-key"
```

Option C: Vertex AI (Gemini)

```
{
  "llm": {
    "provider": "vertex",
    "model": "gemini-2.5-pro",
    "vertexProjectId": "your-gcp-project-id",
    "vertexLocation": "us-central1",
    "maxTokens": 8192
  },
  "wiki": {
    "rootDir": "/absolute/path/to/my-wiki",
    "sourcesDir": "sources",
    "wikiDir": "wiki",
    "schemaDir": "schema"
  },
  "obsidian": {
    "enabled": true
  }
}
```

Then authenticate with Google Cloud:

```
gcloud auth application-default login
```

No API key is needed — Vertex AI uses Application Default Credentials (ADC).

3. Verify the Setup

```
# Check that config is valid (will throw if misconfigured)
wiki status
```

Environment Variables

All configuration can be set via environment variables, which take priority over `config.json`.

Variable	Required	Description
`WIKI_LLM_PROVIDER`	Yes	`anthropic`, `azure`, or `vertex`
`WIKI_LLM_MODEL`	Yes	Model identifier

Variable	Required	Description
`WIKI_LLM_API_KEY`	Anthropic/Azure only	API authentication key
`WIKI_LLM_MAX_TOKENS`	Yes	Max output tokens per call
`WIKI_ROOT_DIR`	Yes	Absolute path to wiki root
`WIKI_AZURE_ENDPOINT`	Azure only	Azure AI endpoint URL
`WIKI_AZURE_DEPLOYMENT`	Azure only	Azure deployment name
`WIKI_VERTEX_PROJECT_ID`	Vertex only	GCP project ID
`WIKI_VERTEX_LOCATION`	Vertex only	GCP region (e.g., `us-central1`)

Priority order: CLI arguments > Environment variables > config.json

---

Running in Development

```
# Run commands directly from source (no build needed)
npx tsx src/cli.ts init
npx tsx src/cli.ts ingest article.md
npx tsx src/cli.ts query "What is X?"
npx tsx src/cli.ts lint
npx tsx src/cli.ts status
```

Running from Built Distribution

```
# Build once
npm run build

# Run from dist/
node dist/cli.js init
node dist/cli.js ingest article.md

# Or if globally linked:
wiki init
wiki ingest article.md
```

Obsidian Integration

LLM Wiki generates Obsidian-compatible markdown. To use with Obsidian:

1. Set `wiki.rootDir` to your Obsidian vault path (or a subfolder within it)
2. Set `obsidian.enabled` to `true` in config
3. Open the vault in Obsidian — pages appear with wiki-links, frontmatter, and aliases

Recommended Obsidian plugins:

- **Dataview** — query wiki page frontmatter (tags, types, dates)
- **Graph View** (built-in) — visualize page interconnections
- **Web Clipper** — clip web articles as markdown sources for ingestion

---

Git Version Control

The wiki directory is designed to be a git repository:

```
cd ~/my-wiki
git init
git add .
git commit -m "Initial wiki setup"
```

The `.gitignore` created by `wiki init` excludes `config.json` (may contain API keys) and `node_modules/`.

After each ingest, the CLI suggests a commit message. You can commit manually:

```
wiki ingest new-article.md
git add .
git commit -m "Ingest: new-article.md — 3 pages created, 2 updated"
```

Usage Workflow

Ingest Sources

```
# Single file
wiki ingest path/to/article.md

# Directory (recursive)
wiki ingest path/to/docs/ --recursive

# With tags
wiki ingest paper.pdf --tags research ml

# Dry run (see what would happen)
wiki ingest article.md --dry-run
```

Query the Wiki

```
# Ask a question
wiki query "What are the main themes across all sources?"

# Save the answer as a wiki page
wiki query --save "Compare approach A and approach B"
```

Maintain Wiki Health

```
# Run health checks
wiki lint

# Auto-fix issues (broken links, orphan pages)
wiki lint --fix

# Check specific category
wiki lint --category orphans
```

Manage Sources

```
# List all sources
wiki list-sources

# Remove a source and its pages
wiki remove-source article-name

# Rebuild index from scratch
wiki rebuild-index

# View statistics
wiki status
```

Supported Source Formats

Format	Extensions	Processing
Markdown	`.md`	Read as text
Plain text	`.txt`	Read as text
PDF	`.pdf`	Text extraction via pdf-parse
JSON	`.json`	Parsed and stringified
CSV	`.csv`	Read as text
Images	`.png`, `.jpg`, `.jpeg`, `.webp`	Base64 encoded, processed via LLM vision

---

Troubleshooting

Configuration Errors

```
ConfigurationError: Missing required configuration field: llm.apiKey
```

Every required field must be set — there are no default values. Check `docs/design/configuration-guide.md` for the full list of fields per provider.

API Key Expiry Warning

```
[WARN] API key expires in 5 days (2026-04-14). Renew your API key soon.
```

Set `llm.apiKeyExpiry` in config.json to enable proactive warnings. Renew the key before it expires.

Vertex AI Authentication

```
Error: Could not load the default credentials
```

Run `gcloud auth application-default login` to authenticate, or set `GOOGLE_APPLICATION_CREDENTIALS` to a service account key file path.

Azure Endpoint Issues

```
Error: Azure AI request failed (status 401): Unauthorized
```

Verify `azureEndpoint` URL and `apiKey` in your config. Ensure the deployment name matches an active deployment in your Azure AI resource.

Build Errors

```
# Type check
npm run typecheck

# Full build
npm run build

# Run tests
npm test
```

Project Structure (Source)

```
wiki/
  src/
    cli.ts                  # CLI entry point (commander)
    commands/               # 8 CLI commands
    config/                 # Config loading, validation, types
    llm/                    # LLM providers (Anthropic, Azure, Vertex)
    wiki/                   # Wiki page management, index, log, registry
    source/                 # Source reading, hashing, chunking
    ingest/                 # Ingest pipeline (summarize, extract, merge)
    query/                  # Query pipeline
    lint/                   # Lint checks (structural + semantic)
    utils/                  # Logger, naming utilities
    templates/              # Default config, schema, prompt templates
  test_scripts/             # 11 test files (180 tests)
  docs/
    design/                 # Project design, plans, config guide, this
file
    reference/              # Investigation docs, research, codebase scans
```